

TabDDPM (Tabular Denoising Diffusion Probabilistic Model): Generating Synthetic Patient Data with Diffusion Models for Imbalanced Clinical Classification

What Problem Is It Solving

Medical datasets have class imbalance. In the Diabetes dataset, only 11% of patients were readmitted. A classifier trained on this data learns to ignore the rare class because it's barely seen it. The model misses the exact patients that matter most clinically. TabDDPM fixes this by generating synthetic minority class patients to balance the training data.

What Is The Generative Deep Learning

A diffusion model. It learns the distribution of real patient records by training on a noise-removal task — given a noisy version of a patient row, predict the noise that was added. At generation time you start from pure random noise and run the reverse process step by step until you get a realistic patient row. You condition this on a class label so you can say "generate me a readmission patient specifically." That generation of new tabular rows from learned distribution is the GDL.

What Is The Input

A row from the dataset. 50 columns per patient. Mix of continuous columns like age, number of medications, time in hospital — and categorical columns like gender, race, insulin status, A1C result, diagnosis codes. You feed this mixed-type tabular data into the diffusion model during training.

What Is The Output

Synthetic patient rows. Fake patients that never existed, generated by the trained diffusion model, conditioned on the readmission label. You add these to your training set and retrain a classifier. The output you present is a comparison table — F1 score on the minority class across: no augmentation vs. SMOTE vs. CTGAN vs. TabDDPM.

What Paper Is It Based On

Primary paper — TabDDPM: Modelling Tabular Data with Diffusion Models, Kotelnikov et al., ICML 2023 <https://arxiv.org/abs/2209.15421>

Foundational paper — DDPM: Denoising Diffusion Probabilistic Models, Ho et al., NeurIPS 2020
<https://arxiv.org/abs/2006.11239>

What Dataset

Diabetes 130-US Hospitals dataset. 101,766 real patient records. 50 features. Free on Kaggle, downloads in seconds, zero credentials needed. Natural class imbalance — 11% readmitted, 89% not.

<https://www.kaggle.com/datasets/jimschacko/airlines-dataset-to-predict-a-delay> — search "diabetes 130 hospital readmission" on Kaggle directly.

What Tech Stack

- PyTorch — implementing TabDDPM
- pandas / NumPy — data loading and preprocessing
- scikit-learn — classifier (XGBoost or logistic regression), F1 and AUC computation
- imbalanced-learn — SMOTE baseline (one line of code) <https://imbalanced-learn.org>
- SDV library — CTGAN baseline (one line of code) <https://github.com/sdv-dev/CTGAN>
- matplotlib / seaborn — all plots and visualizations
- scipy — Jensen-Shannon divergence for fidelity evaluation
- Google Colab — runs everything, free, no GPU needed

One install command: `pip install torch pandas numpy scikit-learn imbalanced-learn sdv matplotlib seaborn scipy`

What Is The Scope

You implement TabDDPM in PyTorch from the paper. Train it on the diabetes dataset. Generate synthetic minority class patients. Augment the training set. Retrain a downstream classifier. Compare F1 on the rare class against three baselines — no augmentation, SMOTE, and CTGAN. Evaluate fidelity of generated data with distribution plots and t-SNE. Run the experiment across 3 augmentation ratios (50%, 100%, 200% of minority class size).

What Is The Approach

Week 1–2: Download data, explore it, measure class imbalance, set up GitHub. Week 3–4: Implement and run all three baselines. Record baseline F1 numbers. Week 5–7: Implement

TabDDPM in PyTorch. Handle continuous columns with Gaussian diffusion, categorical columns with multinomial diffusion. Add class conditioning. Train. Week 8–9: Generate synthetic patients. Evaluate fidelity. Week 10–11: Run full comparison experiments. Write interim report. Week 12–13: Ablation studies, failure analysis, final report. Week 14–15: Presentation prep, code cleanup, submit by May 7.

Code References

TabDDPM official PyTorch code: <https://github.com/rotot0/tab-ddpm> This is public, well written, and directly runnable. You adapt this rather than writing from scratch.

How Easy or Difficult Compared To GANITE

Easier in almost every way that matters practically.

Training stability — TabDDPM uses a simple regression loss (predict the noise). Loss decreases smoothly every run. GANITE uses two competing GANs which can collapse, diverge, or produce garbage without warning. This is the biggest practical difference for your team.

Architecture complexity — TabDDPM is one network doing one job. GANITE is two GANs chained together where Block 1 feeds into Block 2. Understanding that connection and defending it in Q&A is harder.

Debugging — with a stable loss curve you always know if training is working. With GANITE you often can't tell.

Evaluation — F1 score comparison is immediately understandable to any audience. PEHE requires explaining counterfactual theory before the metric even makes sense.

Conceptual depth — GANITE requires understanding causal inference, counterfactuals, and why generated data can supervise a second network. TabDDPM requires understanding diffusion which is conceptually cleaner — add noise, learn to remove it, generate by removing from random noise.

The one place TabDDPM is slightly harder — handling mixed data types (continuous and categorical in the same model) adds preprocessing complexity GANITE doesn't have. But this is a data engineering problem, not a deep learning problem, and it's solved once at the start.

Overall: TabDDPM is meaningfully easier to implement correctly, significantly more stable to train, and produces a cleaner evaluation story. GANITE has deeper GDL roots conceptually but higher execution risk for a weaker team.